Operating Systems

Dmitry Zaitsev

Lecture 13: Building libraries and applications from source code in Linux. Case study: LAPACK (LINPACK)

Typical workflow in general

- Download a tarball (tar.gz or tar.bz2 file), which is a release of a specific version of the source code
- Extract the tarball with a command like tar zxvf myapp.tar.gz for a gzipped tarball or tar jxvf myapp.tar.bz2 for a bzipped tarball
- cd into the directory created above
- run ./configure && make && sudo make install

Install from GitHub or Other Repositories

- Use *git* or *svn* or whatever to pull the latest source code from their official source repository
- cd into the directory created above
- run ./autogen.sh && make && sudo make install

Basic Components

- Source code of an application or system
- Configuration scripts and configuration parameters – shell variables
- *Makefile* generated by the configuration scripts
- *make* to compile and link
- *sudo make install* to provide access for everyone on the machine
- basic targets of *make*: build, install, clean
- *make install* creates the require public directiries and moves there applications and libraries

Public repositories for open source software

- Basic repositories: GitHUb, GitLab, Bitbucket, Codeberg, SourceForge, SourceHut, etc
- Version Control: Repositories track the complete historical timeline of a project's evolution.
- Branching & Merging: Developers can create "branches," which are isolated copies of the codebase, to work on new features or bug fixes without affecting the main production code.
- Documentation: README.md (project overview), LICENSE (usage rights), and CONTRIBUTING.md
- Automation & Security: Vulnerability Scanning, Release Management

Case Study
Solve linear system with LAPACK

https://dimazaitsev.github.io

Basic stages of numerical solving application tasks

- General statement of the problem
- Systems of differential equations and partial differential equations
- Finite difference and finite element method
- System of linear algebraic equations
- Solution or basis of solutions

System of linear algebraic equations

$$\begin{cases} 2x_1 + 5x_2 + 8x_3 = 13 \\ 3x_1 - 6x_2 + 9x_3 = 25 \\ 4x_1 + 7x_2 - 1x_3 = -7 \end{cases}$$

$$x_1 \approx 0{,}302; \quad x_2 \approx -0{,}873;$$

$$x_3 \approx 2{,}095$$

Matrix representation

$$A\vec{x} = \vec{b}$$

$$A = \begin{pmatrix} 2 & 5 & 8 \\ 3 & -6 & 9 \\ 4 & 7 & -1 \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 13 \\ 25 \\ -7 \end{pmatrix}$$

Solution of linear systems

Homogenous:

$$A\vec{x} = 0, \;\; \vec{x} = G\vec{y}$$

Heterogeneous:

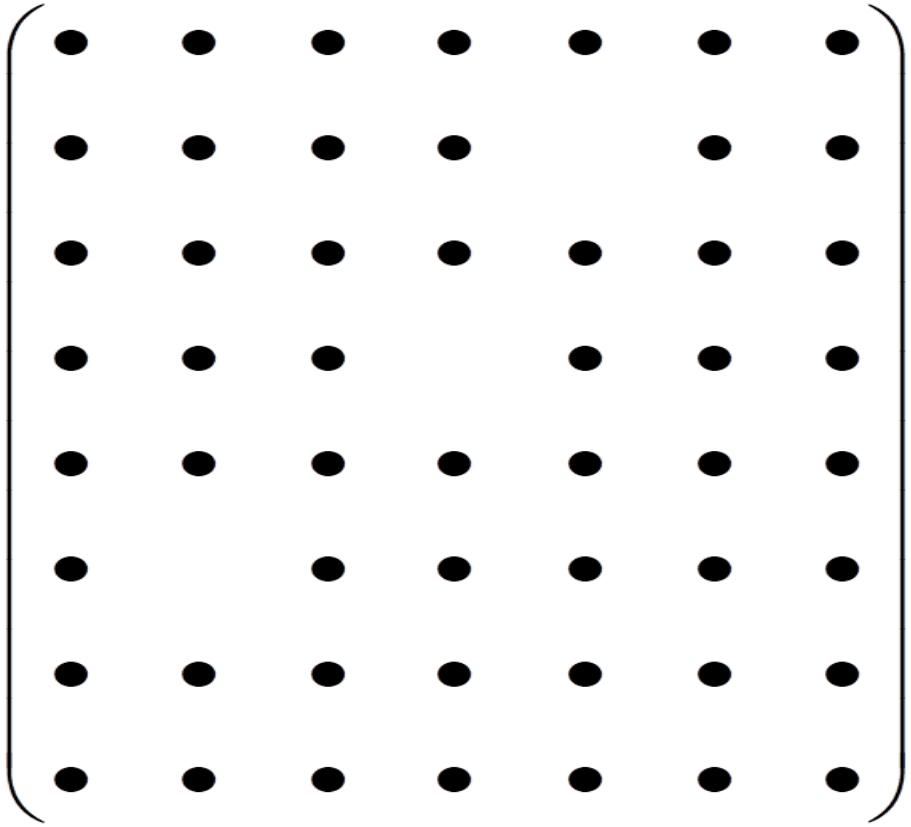
$$A\vec{x} = \vec{b}, \;\; \vec{x} = \vec{x}' + G\vec{y}$$

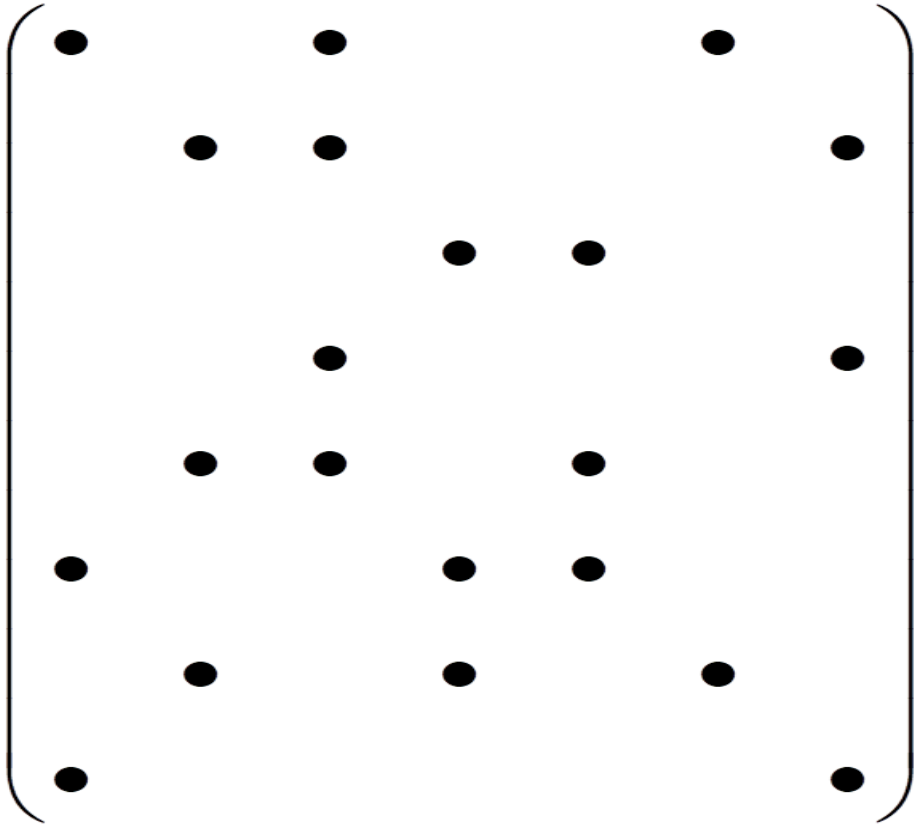
G – matrix of basis solutions

Algebraic structure

Numbers	Example	Structure	Methods
Complex	-3,2+6,25i	field	а) reduction: LU, QR; б) iteration methods
Real	0,25; -78,931		
Integer	-33; 0; 6	ring	Normal forms: Hermite, Smith
Nonnegative integer	0; 7; 55	monoid	Methods of Toudic (Silva) and Contejean

Dense and sparse systems





Practical value

- Numerical methods of solving differential equations and systems and also partial differential equations and systems
- Domains: thermodynamics, weather forecast, trajectories of moving objects, fluid dynamics, nuclear physics etc
- Diophantine systems: artificial intelligence, cryptography, model-checking etc

Real-life matrices (systems)

- Matrix Market - https://math.nist.gov/MatrixMarket
- The SuiteSparse Matrix Collection https://sparse.tamu.edu
- Model Checking Contest – Petri net models https://mcc.lip6.fr

Basic software

Structure \ Type	Dense	Sparse
Field	LAPACK	UMFPACK
Ring	4ti2	ParAd
Monoid	4ti2	ParAd

LAPACK (**L**inear **A**lgebra **Pack**age)

- http://www.netlib.org/lapack
- Legend of Jack Dongarra (1980) http://icl.utk.edu
- Most widespread package
- Most cited computer science paper
- Performance test of supercomputers http://top500.org

LAPACK on parallel architecture

Multicore	Distributed nodes	GPUs
OpenMP	MPI, PVM	CUDA
PLASMA	ScaLAPACK DPLASMA	MAGMA
http://icl.utk.edu		

What is LAPACK

- An open-source library containing methods for solving basic linear algebra problems. It is written in Fortran using another library, BLAS, and is an extension of the LINPACK package.
- There are also analogues that retain or even expand functionality and provide higher performance:
  - Intel® MKL[1]
  - AMD™ ACML [2]
  - Sun Performance Library[3]
  - NAG’s LAPACK[4]
  - HP’s MLIB[5]

BLAS routines: BLAS-1: vector operations

Name	Description
copy	copy vector, y = x
scal	scale vector, y = alpha*y
swap	swap two vectors, y <—> x
axpy	y = alpha*x + y
nrm2	vector 2-norm
amax	vector max-norm
asum	vector one-norm
dot	dot product (real), x^T y
dotu	dot product (complex), unconjugated, x^T y
dotc	dot product (complex), conjugated, x^H y

BLAS-2: matrix-vector operations

Name	Description
gemv	general matrix-vector product, y = A*x
symv, hemv	symmetric/Hermitian matrix-vector product, y = A*x
syr, her	symmetric/Hermitian rank-1 update, A = A + x*x^H
syr2, her2	symmetric/Hermitian rank-2 update, A = A + x*y^H + y*x^H
trmv	triangular matrix-vector product, y = A*x
trsv	triangular solve, one right-hand side (RHS), solve Ax = b

BLAS-3: matrix-matrix operations

Name	Description
gemm	general matrix-matrix multiply, C = C + A*B
symm, hemm	symmetric/Hermitian matrix-matrix multiply, C = C + A*B, A is symmetric
syrk, herk	symmetric/Hermitian rank-k update, C = C + A*A^H, C is symmetric
syr2k, her2k	symmetric/Hermitian rank-2k update, C = C + A*B^H + B*A^H, C is symmetric
trmm	triangular matrix-matrix multiply, B = A*B or B*A, A is triangular
trsm	triangular solve, multiple RHS, solve A*X = B or X*A = B, A is triangular

Lapack functions

sgesv.f
Solves a general system of linear equations AX=B.
sgels.f
Computes the least squares solution to an over-determined system , of linear equations, A X=B or A**H X=B, or the minimum norm , solution of an under-determined system, where A is a general , rectangular matrix of full rank, using a QR or LQ factorization , of A.
sgees.f
Computes the eigenvalues and Schur factorization of a general , matrix, and orders the factorization so that selected eigenvalues , are at the top left of the Schur form.
sgesvd.f
Computes the singular value decomposition (SVD) of a general , rectangular matrix.
…

Data type prefix

s	real	float
d	double precision	double
c	complex	float complex
z	complex*16	double complex

Matrix type prefix

	full	banded	packed	tridiag	generalized problem
general	ge	gb		gt	gg
symmetric	sy	sb	sp	st	
Hermitian	he	hb	hp	ht	
SPD/HPD	po	pb	pp	pt	
triangular	tr	tb	tp		tg
upper Hessenberg	hs				hg
trapezoidal	tz				
orthogonal	or	op			
unitary	un	up			
diagonal	di				
bidiagonal	bd				

Solvers

Types				Routine	F95	MAGMA		Matrix type	Description
Linear systems — LU — driver									
s	d	c	z	gesv	F95	cpu	gpu	general	solve Ax = b
	ds		zc	gesv			gpu	general	solve Ax = b, mixed precision
s	d	c	z	gesvx	F95			general	solve Ax = b, expert
Linear systems — LU, banded — driver									
s	d	c	z	gbsv	F95			general banded	solve Ax = b
s	d	c	z	gbsvx	F95			general banded	solve Ax = b, expert
Linear systems — LU, tridiagonal — driver									
s	d	c	z	gtsv	F95			general tridiagonal	solve Ax = b
s	d	c	z	gtsvx	F95			general tridiagonal	solve Ax = b, expert

lapacke.h

- lapack_int LAPACKE_sgesv(

```
    int matrix_layout,
    lapack_int n,
    lapack_int nrhs,
    float* a,
    lapack_int lda,
    lapack_int* ipiv,
    float* b,
    lapack_int ldb );
```

Program in C

```
#include <stdio.h>
#include <lapacke.h>
#define size 3    /* dimension of matrix */
int main(){
  lapack_int n=size, nrhs=1, lda=size, ldb=1,
             info, ipiv[size];
  float A[size*size] = {2.0,5.0,8.0,
                        3.0,-6.0,9.0,
                        4.0,7.0,-1.0};
  float b[size] =      {13.0,25.0,-7.0};
  int i,j;
  info = LAPACKE_sgesv(LAPACK_ROW_MAJOR, n, nrhs,
                       A, lda, ipiv, b, ldb);
  for (i=0; i<n; i++) printf("%f\n", b[i]);
}
```

Makefile

```
include ../../make.inc

all: lp_ex1

LIBRARIES = ../../$(LAPACKELIB) ../../$(LAPACKLIB) $(BLASLIB)

lp_ex1: lp_ex1.o $(LIBRARIES)
	$(LOADER) $(LOADOPTS) -o $@ $^
	./$@

clean: cleanobj cleanexe
cleanobj:
	rm -f *.o
cleanexe:
	rm -f x*

.c.o:
	$(CC) $(CFLAGS) -I. -I../include -c -o $@ $<
```

Run example

```
hare@hare-world:~/lapack-3.8.0/LAPACKE/example$ make
gcc -O3 -I. -I../include -c -o lp_ex1.o lp_ex1.c
gfortran -o lp_ex1 lp_ex1.o
                    ../../liblapacke.a ../../liblapack.a ../../librefblas.a
./lp_ex1
 0.301587
-0.873016
2.095238
hare@hare-world:~/lapack-3.8.0/LAPACKE/example$
```

Use LAPACK to benchmark computer hardware and operating systems

```
$ time ./xexample_DGESV_rowmajor -n 5000 > /dev/null

real    0m47.094s
user    0m46.725s
sys     0m0.336s
```